



DIRECTION GÉNÉRALE DES ENTREPRISES

Architecture du code

Pour la personne qui doit reprendre, auditer ou héberger l'application

MARI(a)SOL — Mapping d'Alignement Réglementaire par Intelligence Albert pour le Suivi des Orientations Législatives

Version 1.5.3 du 9 septembre 2026

Document établi le 02/10/2026

Ce document s'adresse à la personne qui doit **reprendre, auditer ou héberger** l'application. Il dit ce que fait chaque fichier, pourquoi il existe, et ce qu'il faut savoir avant d'y toucher. Il ne remplace pas le code : chaque module porte en tête une explication de ses choix, et c'est là que se trouve le détail.

1. Le principe qui commande tout le reste

L'application ne connaît que les documents qu'on y charge. Elle ne complète jamais avec des connaissances extérieures. Cette contrainte explique la forme du code :

- **Le calcul est déterministe, le modèle ne fait que qualifier.** Matrices, taux d'accord, blocs, majorité qualifiée, écarts entre versions, lecture d'un acte modificatif : tout cela est de l'arithmétique et des expressions régulières. Le modèle de langage n'intervient que pour dire ce qu'un écart déjà constaté signifie, jamais pour dire qu'il y a un écart.
- **Aucune affirmation sans citation vérifiée.** Toute phrase produite par le modèle porte une citation, et le programme la recherche littéralement dans le document source. Si elle ne s'y trouve pas, l'affirmation est **retirée**, pas signalée.
- **Rien ne sort du poste, sauf vers Albert.** Les documents restent dans `data/`. Seuls les extraits nécessaires à une analyse partent vers l'API Albert de la DINUM, hébergée en SecNumCloud. Aucun autre service n'est appelé.

2. Vue d'ensemble

<code>app.py</code>	point d'entrée Streamlit, navigation, bandeau latéral
<code>accueil.py</code>	page d'accueil
<code>pages/</code>	un fichier par écran (9 écrans)
<code>core/</code>	toute la logique, sans aucun appel Streamlit
<code>data/</code>	corpus, base SQLite, exports – jamais versionné
<code>assets/</code>	logo et déclinaisons
<code>documentation/</code>	les guides du dossier, en PDF
<code>tests/</code>	116 tests automatiques

Les PDF de `documentation/` sont fabriqués à partir de sources Markdown par `tools/build_docs.py`, qui reste dans la copie de développement : le dossier remis ne porte que les PDF, pour n'avoir qu'une version de chaque document en circulation.

La séparation `pages/ ↔ core/` est stricte et volontaire : **aucun module de `core/` n'importe Streamlit**. C'est ce qui rend chaque calcul testable sans interface, et c'est ce qui permettrait de servir la même logique autrement qu'en Streamlit sans rien réécrire.

La chaîne de traitement, de bout en bout :

```

fichier PDF/DOCX
  ↓ ingest.py          lecture, détection du type, découpage en sections
  ↓ wk_parser.py       (si tableau WK) une contribution par État membre
  ↓ store.py           écriture SQLite + index plein texte FTSS
  ↓ scoring.py         classement de chaque contribution par le modèle
  ↓ coalition.py       qui pense comme qui, arithmétique du Conseil
  ↓ report.py / excel.py note Word, classeur, images

```

3. Le socle : `core/`

Configuration et infrastructure

Fichier	Ce qu'il fait
<code>config.py</code>	Toute la configuration, pilotée par variables d'environnement <code>REGWATCH_*</code> et <code>ALBERT_*</code> . Porte le numéro de version affiché à l'écran, le mode démonstration (qui verrouille la clé API), et <code>chemin_affiche()</code> , qui n'affiche jamais un chemin absolu — l'application tourne sur des postes dont l'arborescence diffère.
<code>store.py</code>	Persistance SQLite : documents, segments, contributions, analyses, positions françaises de référence, journal d'audit. Contient les migrations de schéma (<code>PRAGMA user_version</code>), la sauvegarde et la restauration à chaud, et le contrôle d'intégrité de l'index. C'est le seul module qui écrit sur le disque.
<code>llm.py</code>	Client de modèle, sous contrainte. Impose un schéma de sortie, réessaie, et convertit toute défaillance réseau en une erreur typée que les pages savent afficher. Trois fournisseurs : Albert (défaut), tout point d'accès compatible OpenAI, et un mode hors ligne qui n'appelle rien.
<code>schemas.py</code>	Les contrats de sortie imposés au modèle, en Pydantic. Un modèle qui répond hors du schéma est rejeté, pas rattrapé.
<code>palette.py</code>	Les deux conventions de couleur — accessible (bleu/rouge, lisible en vision daltonienne) et note de direction (vert/jaune/rouge) — en un seul endroit, pour que l'écran, le PNG, le Word et l'Excel disent la même chose.
<code>duree.py</code>	Estimation et affichage des durées d'une opération longue. Une analyse de 700 contributions dure une demi-heure : le dire avant de la lancer est une fonctionnalité.

Lecture des documents

Fichier	Ce qu'il fait
<code>ingest.py</code>	Ingestion universelle. Lit un PDF ou un DOCX, détecte le type de document (tableau WK du Conseil, texte réglementaire, texte libre), découpe en sections — exposé des motifs, considérants, articles, annexes — et subdivise les articles longs. Traite à part les actes modificatifs , découpés instruction par instruction plutôt qu'article par article. Retire le pied de page courant des documents européens (« FR FR 23 »), qui sinon se colle à la fin des phrases.
<code>wk_parser.py</code>	Lecture des tableaux de commentaires consolidés du Conseil : une ligne par contribution, avec l'État membre, l'article visé, et la distinction entre commentaire de fond et proposition de rédaction.

Fichier	Ce qu'il fait
<code>nonpaper.py</code>	Extraction de positions depuis un document libre : non-paper, papier blanc, compte rendu de réunion. Devine l'auteur, et n'extrait une position que lorsqu'il y en a une — un extrait de politesse ou de procédure n'en produit aucune.
<code>eurlex.py</code>	Récupération des textes publics depuis EUR-Lex, en HTML et non en PDF : le PDF du Journal officiel est une mise en page à deux colonnes dont l'extraction introduit des écarts qui se compteraient ensuite comme des modifications. Contient aussi le catalogue des noms d'usage (CELEX → « RGPD », « Data Act ») et la dérivation d'un nom lisible depuis le document lui-même.
<code>labels.py</code>	Intitulés lisibles. « Article 24 (2/12) » ne renvoie à rien dans le texte officiel ; ce module le remplace par la première disposition réellement contenue dans le fragment.
<code>titres.py</code>	Regroupement des articles par titre du règlement, à partir des en-têtes lus dans le texte ou de bornes saisies à la main.

Analyse

Fichier	Ce qu'il fait
<code>retrieval.py</code>	Index plein texte SQLite FTS5 , en table autonome, avec classement BM25. Sert la recherche et la sélection des passages soumis au modèle.
<code>qa.py</code>	Réponse sourcée : chaque affirmation porte une citation, et la citation est vérifiée littéralement dans le document source. Les affirmations non vérifiées sont supprimées, et leur nombre est affiché.
<code>scoring.py</code>	Le cœur de la cartographie. Classe chaque contribution — alignée, partiellement alignée, divergente — par rapport à un référentiel : la position française, ou le texte initial. Applique la règle du silence : sur un article que la France n'a pas amendé, la référence est le maintien du texte en l'état. Un appel de modèle par contribution, jamais un lot : un modèle à qui l'on donne quinze contributions répond sur la première et oublie les autres.
<code>coalition.py</code>	Taux d'accord paire à paire, détection de blocs, et arithmétique du Conseil : majorité qualifiée (15 États sur 27 et 65 % de la population) et minorité de blocage (au moins 4 États et plus de 35 %). Les chiffres de population ne servent qu'à cela.
<code>ponderation.py</code>	Pondération d'un critère écrit en français par l'agent, pour reclasser les États membres selon ce qui compte dans le dossier.
<code>diff.py</code>	Comparaison de deux versions d'un même texte : appariement des articles, mesure de similarité, et qualification de la portée d'un écart. L'appariement et la mesure sont déterministes ; seule la qualification appelle le modèle.
<code>modificatif.py</code>	La lecture d'un omnibus. Un acte modificatif ne récrit pas le Data Act : il dit ce qu'il faut y changer. Ce module lit les instructions de modification, les range par texte cible puis par article visé, reconstitue chaque article avant et après quand le texte consolidé est chargé, produit la synthèse transversale de l'acte, et compare deux versions d'un même omnibus instruction par instruction. Entièrement déterministe, sauf la qualification de portée. C'est le plus gros module du socle, et le plus commenté.
<code>amendements.py</code>	« Nos amendements ont-ils été retenus ? » — suivi d'une liste d'amendements dans un texte d'arrivée, avec un verdict par amendement et le refus explicite de conclure quand la preuve manque.

Fichier	Ce qu'il fait
<code>fiabilite.py</code>	Ce qui, dans un résultat affiché, demande l'œil de l'agent : contributions non analysées, citations introuvables, articles documentés par trop peu d'États, réserves d'examen. Ces alertes vivent dans un panneau à part et ne partent jamais dans un livrable : une note de direction n'a pas à porter les doutes de l'outil, l'analyste si.
<code>aide.py</code>	La base de connaissance de l'application : une trentaine de fiches écrites à la main, chacune en deux niveaux — l'explication, et le détail d'implémentation (formules, seuils, bibliothèques). L'assistant du mode d'emploi répond à partir de ces fiches et de rien d'autre. Il ne connaît ni le droit européen ni les documents chargés, et le dit.

Restitution

Fichier	Ce qu'il fait
<code>viz.py</code>	Les graphiques interactifs (Plotly) : matrice d'alignement, classement des alliés, nuage des articles clivants, graphe des blocs, jauge de majorité qualifiée.
<code>report.py</code>	Les livrables hors écran : images PNG (matplotlib, et non l'export d'image de Plotly, qui exige un navigateur Chrome absent d'un poste d'administration), note Word de cartographie, fiche d'orientation entreprise, et note de synthèse sur un acte modificatif . Tous les tableaux sont de vrais tableaux Word, modifiables, jamais des captures.
<code>excel.py</code>	Le classeur de restitution, avec ses graphiques liés aux cellules et sa feuille de détail. Reprend exactement les teintes de la palette active, pour se poser à côté du suivi tenu à la main sans que l'œil ait à réapprendre le code couleur.
<code>ui.py</code>	Les éléments d'interface partagés : en-tête de module, fenêtre « Comment ça marche ? », panneau de fiabilité, tableau de suivi coloré, et la carte d'un acte modificatif. C'est le seul endroit de l'application où de l'affichage est écrit à la main en HTML, et la raison en est donnée sur place.

4. Les écrans : `pages/`

Chaque fichier de `pages/` est un écran. Aucun ne contient de calcul : ils appellent `core/`, affichent, et rendent la main.

Écran	Ce qu'on y fait
<code>0_Mode_d_emploi.py</code>	Comment se servir de l'outil, en quatre onglets, plus un assistant qui répond sur le fonctionnement de l'application à partir des fiches de <code>aide.py</code> .
<code>1_Bibliotheque.py</code>	Charger des documents, récupérer un texte public depuis EUR-Lex, corriger un type ou un repère de version. Un fichier illisible ne fait plus échouer tout le lot.
<code>2_Recherche.py</code>	Une question en français, une réponse dont chaque affirmation cite un passage retrouvé littéralement. Deux modes : réponse sourcée, ou extraits bruts sans appel de modèle.

Écran	Ce qu'on y fait
<code>3_Positions_Etats_membres.py</code>	Le cœur de l'outil : la matrice d'alignement, le tableau de suivi coloré, les alliés, les articles clivants, le détail contribution par contribution, l'apport des non-papers, et les exports Word et Excel.
<code>4_Coalitions.py</code>	Qui aller chercher, quels blocs se forment sans nous, le détail paire par paire, et l'arithmétique du Conseil.
<code>5_Comparaison_de_versions.py</code>	Le fil d'un texte à travers ses versions, les changements majeurs triés, les cartes, le détail mot à mot — et l'onglet Omnibus et actes modificatifs , en trois vues.
<code>6_Suivi_des_amendements.py</code>	Le sort de nos amendements dans un texte d'arrivée. Refuse un acte modificatif comme texte d'arrivée plutôt que de rendre des verdicts faux.
<code>7_Regimes_d_acces.py</code>	Une situation d'entreprise décrite par un formulaire, et ce que les textes chargés en disent, disposition par disposition. Produit une fiche d'orientation Word.
<code>8_Administration.py</code>	Clé API et modèle, données de référence, état et réparation de la base, sauvegarde et restauration, journal d'audit.

5. Ce qui n'est pas du code

Fichier	À quoi il sert
<code>demarrer.bat / demarrer.sh</code>	Démarrage en un double-clic. Créent l'environnement virtuel, installent les dépendances, lancent l'application. Détectent le Python de la boutique Microsoft, gèrent un chemin réseau UNC, et suppriment un environnement virtuel resté incomplet après une installation interrompue — sans quoi l'outil ne redémarrerait plus jamais.
<code>requirements.txt</code>	Les dépendances, à version exacte . Un intervalle laisse une mise à jour changer le comportement de l'outil entre deux postes, ce qui est exactement ce qu'un déploiement ne doit pas permettre.
<code>.env.example</code>	Le gabarit de configuration. La clé API n'est jamais dans le dépôt.
<code>.streamlit/ config.toml</code>	Le thème et le comportement du serveur local.
<code>tests/</code>	116 tests : le garde-fou des citations, le calcul des coalitions, la lecture des omnibus, les livrables, et quatre tests qui exercent l'écran lui-même sans navigateur.

6. Les dépendances, et pourquoi chacune

Bibliothèque	Rôle	Pourquoi elle plutôt qu'une autre
<code>streamlit</code>	Interface	Une application web sans serveur à administrer ni JavaScript à maintenir.
<code>pandas</code>	Tableaux	Le format d'échange entre tous les modules.
<code>pdfplumber</code>	Lecture PDF	Conserve la structure en lignes, nécessaire au découpage en articles.
<code>python-docx</code>	Word	Produit un document modifiable sur le poste, hors ligne.
<code>openpyxl</code>	Excel	Écrit des graphiques liés aux cellules, pas des images.
<code>matplotlib</code>	Images PNG	Écrit le fichier directement, sans navigateur.
<code>plotly</code>	Graphiques d'écran	Interactif, et sélectionnable pour naviguer d'un graphique vers un article.
<code>networkx</code>	Graphes	Détection des blocs de coalition.
<code>pydantic</code>	Contrats	Impose au modèle un format de sortie vérifié.
<code>openai</code>	Client HTTP	L'API Albert suit la même convention ; aucun code spécifique n'est nécessaire.
<code>sqlite3</code>	Base	Dans la bibliothèque standard. Un fichier, pas un serveur.

Toutes sont des bibliothèques libres, largement diffusées, installables depuis un miroir interne. Aucune dépendance à un service en ligne autre qu'Albert.

7. Ce qu'il faut savoir avant d'y toucher

Ne pas placer `data/` dans un dossier synchronisé. La synchronisation d'un fichier SQLite en cours d'écriture le corrompt. C'est la seule manière connue de perdre des données avec cet outil.

Les variables d'environnement s'appellent `REGWATCH_*`. Le nom d'usage de l'application a changé, pas les variables ni le nom du dossier : une installation existante doit continuer de fonctionner sans que personne ait à toucher son fichier de configuration.

Le mode démonstration verrouille la clé API dans `config.py`, pas dans l'interface. Un écran oublié ne doit pas permettre d'enregistrer une clé qui deviendrait active pour tous les utilisateurs d'une instance partagée.

Une modification de `core/` doit passer les tests. `python -m pytest -q` depuis le dossier de l'application. Le test le plus important est celui du garde-fou des citations : c'est la promesse principale de l'outil.